REACT NATIVE · WEEKS 10 - 12

Complete Study Notes

Definitions · Code Examples · MCQs · 4 Quizzes · Final Term Questions — all in one place.

Week 10 — React Native Intro

Week 11 — Components & Input

Week 11 P2 — FlexBox

Week 12 — Lists & Navigation

Table of Contents

1. **Week 10 — What is React Native?**
   - Definitions & Simple Explanations
   - Code Examples
2. **Week 11 — Basic Components & User Input**
   - Definitions & Simple Explanations
   - Code Examples
3. **Week 11 Part 2 — FlexBox in React Native**
   - Definitions & Simple Explanations
   - Code Examples
4. **Week 12 — ScrollView, FlatList & Navigation**
   - Definitions & Simple Explanations
   - Code Examples
5. **MCQ Bank (All Topics — 24 Questions)**
6. **4 Quizzes (10 Marks Each)**
7. 📌 **Final Term Questions (40% Weight)**

WEEK 10

What is React Native?

① Definitions & Simple Explanations

REACT NATIVE

React Native is a **framework** that lets you build real mobile apps for both Android and iOS using JavaScript and React — the same language used for websites. It is not a web app; it produces genuine native apps that feel smooth and fast.

> **Simple analogy:** *Think of React Native as a translator. You write code in JavaScript (which you already know), and React Native translates it into the language that Android and iOS phones understand natively.*

CROSS-PLATFORM DEVELOPMENT

Writing **one codebase** that runs on both Android and iOS, instead of writing two separate apps. This saves time and money.

> **Example:** *Companies like Facebook, Instagram, and Shopify use React Native so their teams write code once and ship to both app stores.*

REACT.JS VS REACT NATIVE

React.js is for building **websites** that run in a browser. React Native is for building **mobile apps** that run on phones. They share the same logic (components, state, props) but use different building blocks for the UI.

> **Key difference:** *React.js uses HTML tags like* `<div>` *and* `<h1>`*. React*

Native uses components like `<View>` *and* `<Text>`.

Feature	React.js (Web)	React Native (Mobile)
Runs on	Browser	Android & iOS phone
Container element	`<div>`	`<View>`
Text element	`<h1>, <p>`	`<Text>`
Styling	CSS files	JavaScript StyleSheet
Navigation	React Router	React Navigation
Device access	Limited (browser)	Camera, GPS, etc.

EXPO CLI

Expo CLI is the **easy, beginner-friendly** way to create React Native apps. It comes pre-packaged with many tools so you can start building immediately without complex Android Studio or Xcode setup.

Analogy: *Expo CLI is like a ready-made meal kit — everything is already included. React Native CLI is like cooking from scratch — more control, but more work to set up.*

REACT NATIVE CLI

The official, lower-level tool for creating React Native apps. It gives you **full access to native code** (Java/Kotlin for Android, Swift/Objective-C for iOS), which is needed for advanced or custom apps. It requires Android Studio or Xcode to be installed.

EXPO GO

A free app you install on your real phone (from Play Store / App Store). When you run `npx expo start` on your PC, Expo Go scans the QR code and immediately shows your app on your phone — no build step needed.

HOT RELOADING / LIVE RELOADING

Hot Reloading means when you save a code change on your computer, the app on your phone updates *instantly* without restarting. This makes development very fast.

② Code Examples

Setting Up an Expo App (Terminal Commands)

TERMINAL / CMD

```
# Step 1 - Create a new Expo project
npx create-expo-app@latest

# Step 2 - Move into the project folder
cd MyFirstApp

# Step 3 - Start the development server (shows QR code)
npx expo start

# Step 4 - Run on an Android emulator
npm run android

# Step 5 - Run on an iOS simulator
npm run ios
```

Hello World — Your First React Native App

APP.JS — REACT NATIVE

```
import React from 'react';
import { Text, View } from 'react-native';

export default function App() {
  return (
    <View>            // ← acts like <div> in HTML
      <Text>Hello, React Native!</Text>
    </View>
  );
}
```

Project Folder Structure

FILE STRUCTURE

```
MyFirstApp/
├── App.js          ← Your main component (entry point)
├── assets/         ← Images, fonts, icons
├── node_modules/   ← All installed packages (auto-generated)
├── package.json    ← Project info & dependencies list
└── app.json        ← Expo configuration
```

WEEK 11

Basic Components & User Input

① Definitions & Simple Explanations

TYPESCRIPT (BRIEF INTRO)

TypeScript is JavaScript with **type checking**. It catches mistakes in your code *before* you even run it. Files use the `.ts` or `.tsx` extension. You will see it used in Expo projects but you can also write plain JavaScript — it is optional for beginners.

> **Example:** *If you declare a variable as a number, TypeScript will warn you if you accidentally assign a string to it.*

VIEW COMPONENT

`<View>` is the most fundamental building block in React Native. Think of it as the equivalent of `<div>` in HTML. It is a container you use to group and layout other components. By itself it is invisible — it only becomes visible when you give it a background colour or border.

TEXT COMPONENT

`<Text>` is the only component you can use to display text in React Native. Unlike HTML where any tag can hold text, in React Native **all text must be inside a `<Text>` tag**.

> **Wrong:** `<View>Hello</View>` — *this will crash.*
> **Correct:** `<View><Text>Hello</Text></View>`

IMAGE COMPONENT

`<Image>` displays images in your app. You can load images from your local project using `require('./path/to/image.jpg')`, or from the internet using a URL via the `source={{ uri: '...' }}` prop.

BUTTON COMPONENT

The built-in `<Button>` creates a simple clickable button. It requires a `title` prop (the button label) and an `onPress` prop (the function that runs when tapped). Its look is controlled by the operating system, so it looks slightly different on Android vs iOS.

TEXTINPUT COMPONENT

`<TextInput>` is like an HTML `<input>` field — it lets users type text. The `onChangeText` prop fires every time the user types a character, and you save the value using React's `useState`.

> **Important props:** *`placeholder`* *(ghost text when empty),* *`value`* *(controlled value),* *`onChangeText`* *(callback),* *`keyboardType`* *(e.g. "email-address", "numeric").*

KEYBOARDAVOIDINGVIEW

When the user taps a text input, the phone's keyboard pops up and can **cover your input field**. `<KeyboardAvoidingView>` automatically pushes the UI upward to keep the input visible. Use `behavior="padding"` on iOS and `behavior="height"` on Android.

TOUCHABLEOPACITY

A better alternative to `<Button>` when you want a **fully customisable button**. You can wrap any element (text, image, icon) inside it. When tapped, the element briefly becomes semi-transparent, giving visible feedback to the user.

> **Why use it over Button?** *Because you can style it completely — change colour, shape, size, add icons, etc. The default Button cannot be styled that much.*

STYLESHEET.CREATE()

In React Native you don't use CSS files. Instead, you define styles using JavaScript objects, and you pass them to `style={...}` prop. `StyleSheet.create()` is the recommended way because it validates your style properties and slightly improves performance.

② Code Examples

View, Text, Image & Button — Combined Example

REACT NATIVE JSX

```jsx
import React from 'react';
import { View, Text, Image, Button } from 'react-native';

const App = () => {
  return (
    <>
      {/* Text + local image */}
      <View>
        <Text>Welcome to my App</Text>
        <Image
          source={require('./assets/logo.png')}
          style={{ width: 200, height: 200 }}
        />
```

```
      </View>

      {/* Simple button */}
      <View>
        <Button
          title="Press Me"
          onPress={() => alert('Button pressed!')}
          color="#841584"
        />
      </View>
    </>
  );
};

export default App;
```

TextInput — Controlled Input with useState

REACT NATIVE JSX

```
import React, { useState } from 'react';
import { View, TextInput, Text } from 'react-native';

const App = () => {
  const [name, setName] = useState('');  // stores what user types

  return (
    <View style={{ padding: 20 }}>
      <TextInput
        placeholder="Enter your name"
        onChangeText={text => setName(text)} // update state on ev
        value={name}
        style={{ borderColor: 'gray', borderWidth: 1, padding: 10
      />
      <Text>Your name is: {name}</Text>
    </View>
  );
};
```

```
export default App;
```

TouchableOpacity — Custom Styled Button

REACT NATIVE JSX

```
import React from 'react';
import { View, Text, TouchableOpacity, Alert } from 'react-native'

const App = () => (
  <View style={{ marginTop: 100, alignItems: 'center' }}>
    <TouchableOpacity
      onPress={() => Alert.alert('You pressed it!')}
      style={{ backgroundColor: 'blue', padding: 12, borderRadius:
    >
      {/* You can put anything inside — text, icon, image */}
      <Text style={{ color: 'white', fontSize: 16 }}>Click Me</Tex
    </TouchableOpacity>
  </View>
);

export default App;
```

WEEK 11 P2

FlexBox in React Native

① Definitions & Simple Explanations

FLEXBOX

Flexbox (Flexible Box Layout) is the **main layout system** in React Native. It lets you control how components are positioned and sized on the screen — vertically, horizontally, centred, stretched, wrapped, and so on. Every `<View>` is automatically a Flex container — you don't need to write `display: flex` like you would in CSS.

> **Key difference from CSS:** *In web CSS the default direction is `row` (horizontal). In React Native the default is `column` (vertical) — items stack from top to bottom by default.*

FLEX PROPERTY

The `flex` number tells a component how much of the **available space** it should take relative to its siblings. `flex: 1` means "take all available space." If two siblings have `flex: 1` and `flex: 2`, the second takes twice the space.

> **Example:** *Header (flex: 1) + Content (flex: 4) + Footer (flex: 1) — content gets 4/6 of the screen height.*

FLEXDIRECTION

Controls which **direction** children are laid out. `'column'` stacks them top-to-bottom (default). `'row'` places them left-to-right (side by side). `'column-reverse'` and `'row-reverse'` flip the order.

MAIN AXIS VS CROSS AXIS — THE MOST IMPORTANT CONCEPT

Every flex container has two invisible lines running through it. The **Main Axis** runs in the direction of `flexDirection`. The **Cross Axis** runs perpendicular to it. Knowing which axis is which is the key to using `justifyContent` and `alignItems` correctly.

flexDirection	Main Axis	Cross Axis
`column` (default)	Vertical ↕	Horizontal ↔
`row`	Horizontal ↔	Vertical ↕

JUSTIFYCONTENT

Aligns children along the **Main Axis**. Common values: `flex-start` (default — pack at start), `center` (pack in middle), `flex-end`, `space-between` (equal gap between items), `space-around`, `space-evenly`.

> **Quick rule:** *If direction is column, `justifyContent` controls vertical position. If direction is row, it controls horizontal position.*

ALIGNITEMS

Aligns children along the **Cross Axis**. Common values: `flex-start`, `center`, `flex-end`, `stretch` (default — makes children fill the cross-axis width/height).

Quick rule: *If direction is column,* `alignItems` *controls horizontal (left/right) positioning.*

ALIGNSELF

Works the same as `alignItems` but is applied to a **single child** individually, overriding the parent's `alignItems` . Useful when one item should be positioned differently from the rest.

FLEXWRAP

By default, Flex tries to fit all items on one line. Setting `flexWrap: 'wrap'` allows children to **move to the next line** when they run out of space — like how text wraps in a paragraph. Great for grid-like layouts.

SAFEAREAVIEW

Modern phones have notches, dynamic islands, camera holes, and rounded corners. Content placed at the very top or bottom of the screen can be hidden by these. `<SafeAreaView>` automatically adds padding to keep all your content in the "safe" visible area of the screen.

When to use: *Always wrap your root/top-level View in SafeAreaView so your app looks good on all phone models.*

② Code Examples

flex — Dividing Screen Space

REACT NATIVE — FLEX

```
import React from 'react';
import { View } from 'react-native';

export default function App() {
  return (
    <View style={{ flex: 1 }}>  // Takes entire screen

      <View style={{ flex: 1, backgroundColor: 'red' }} />
      {/* Takes 1/4 of screen height */}

      <View style={{ flex: 2, backgroundColor: 'blue' }} />
      {/* Takes 2/4 of screen height (twice as tall) */}

      <View style={{ flex: 1, backgroundColor: 'green' }} />
      {/* Takes 1/4 of screen height */}

    </View>
  );
}
```

justifyContent + alignItems — Perfect Centering

REACT NATIVE — CENTER BOX

```
import React from 'react';
import { View } from 'react-native';

export default function App() {
  return (
    <View style={{
      flex: 1,
      justifyContent: 'center',  // centers on Main Axis (vertical
      alignItems: 'center'       // centers on Cross Axis (horizon
    }}>
      <View style={{ width: 100, height: 100, backgroundColor: 'bl
    </View>
  );
```

```
}
// Output: blue box perfectly centred on screen
```

Full App Layout — Header, Content, Footer

REACT NATIVE — APP LAYOUT

```
import React from 'react';
import { View, Text } from 'react-native';

export default function App() {
  return (
    <View style={{ flex: 1 }}>

      {/* HEADER — 1/6 of screen */}
      <View style={{ flex: 1, backgroundColor: '#3498db',
        justifyContent: 'center', alignItems: 'center' }}>
        <Text style={{ color: 'white', fontSize: 22 }}>Header</Tex
      </View>

      {/* CONTENT — 4/6 of screen */}
      <View style={{ flex: 4, backgroundColor: '#ecf0f1',
        justifyContent: 'center', alignItems: 'center' }}>
        <Text>Main Content</Text>
      </View>

      {/* FOOTER — 1/6 of screen */}
      <View style={{ flex: 1, backgroundColor: '#2ecc71',
        justifyContent: 'center', alignItems: 'center' }}>
        <Text style={{ color: 'white' }}>Footer</Text>
      </View>

    </View>
  );
}
```

WEEK 12

ScrollView, FlatList & Stack Navigation

① Definitions & Simple Explanations

SCROLLVIEW

`<ScrollView>` is a container that makes its content **scrollable** when the content is taller than the screen. It renders all its children at once, which makes it simple to use but slow for very long lists.

> **Best for:** *Short lists, forms, long text (profiles, articles).* **Avoid for:** *Lists of hundreds or thousands of items — use FlatList instead.*

FLATLIST

`<FlatList>` is the **performance-optimised** list component. Unlike ScrollView which loads everything at once, FlatList only renders the items currently *visible* on screen. As you scroll, it loads new items and discards ones that are off-screen. This technique is called **lazy loading / windowing**.

> **Required props:** `data` *(your array),* `renderItem` *(function that returns JSX for each item),* `keyExtractor` *(function that returns a unique key for each item).*

FLATLIST PERFORMANCE PROPS

When your list has thousands of items, you can tune FlatList's performance:

Prop	What it does
`initialNumToRender`	How many items to render on first load
`maxToRenderPerBatch`	How many items to add per scroll batch
`windowSize`	How many screens worth of items to keep in memory
`removeClippedSubviews`	Detaches off-screen views (Android)
`getItemLayout`	Pre-calculates item heights for faster scroll

REACT NAVIGATION

React Navigation is a **third-party library** that handles moving between screens in your app. A mobile app has multiple screens (Home, Profile, Settings) and React Navigation manages how you go from one to another — and provides the back button behaviour.

STACK NAVIGATION

Stack Navigation treats screens like a **stack of cards**. When you navigate to a new screen it is placed on top. When you press "Back" the top card is removed and you see the previous screen again. This is the standard navigation pattern in most apps (e.g. WhatsApp messages list → individual chat → photo viewer).

Key functions: `navigation.navigate('ScreenName')` *— go to a screen,* `navigation.goBack()` *— go back,* `navigation.push('ScreenName')` *— add a new instance even if it already exists in the stack.*

NAVIGATIONCONTAINER

`<NavigationContainer>` is a wrapper component that must wrap your *entire app*. It manages the navigation tree and stores navigation state. Think of it as the "brain" of your navigation system.

CREATENATIVESTACKNAVIGATOR

A function from `@react-navigation/native-stack` that creates a stack navigator. You call it once to get a `Stack` object, then use `<Stack.Navigator>` and `<Stack.Screen>` to define your screens.

② Code Examples

ScrollView — Simple Long List

REACT NATIVE — SCROLLVIEW

```
import React from 'react';
import { ScrollView, View, Text, StyleSheet } from 'react-native';

export default function App() {
  return (
    <ScrollView style={styles.container}>
      {Array.from({ length: 30 }).map((_, index) => (
        <View key={index} style={styles.item}>
          <Text>Item number {index + 1}</Text>
        </View>
      ))}
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20 },
  item: { marginVertical: 10, padding: 20,
```

```
          backgroundColor: '#f2f2f2', borderRadius: 5 }
});
```

FlatList — Optimised for Large Data

REACT NATIVE — FLATLIST

```
import React from 'react';
import { FlatList, View, Text, StyleSheet } from 'react-native';

const App = () => {
  // Create 1000 items to simulate a large dataset
  const largeData = Array.from({ length: 1000 }, (_, i) => ({
    id: i.toString(),
    name: `Item ${i + 1}`
  }));

  // Defines what each row looks like
  const renderItem = ({ item }) => (
    <View style={styles.item}>
      <Text>{item.name}</Text>
    </View>
  );

  return (
    <FlatList
      data={largeData}
      renderItem={renderItem}
      keyExtractor={item => item.id}  // must be unique string
      initialNumToRender={10}           // only render 10 on start
      maxToRenderPerBatch={10}
      windowSize={5}
      removeClippedSubviews={true}     // remove off-screen items
      getItemLayout={(data, index) => (
        { length: 50, offset: 50 * index, index }
      )}
    />
  );
```

```
};
export default App;
```

Stack Navigation — Full Setup

APP.JS — NAVIGATION SETUP

```
// Install packages first:
// npx expo install @react-navigation/native
// npx expo install react-native-screens react-native-safe-area-co
// npx expo install @react-navigation/native-stack

import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/nati
import HomeScreen from './screens/HomeScreen';
import ProfileScreen from './screens/ProfileScreen';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>  // wraps everything
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen name="Home"    component={HomeScreen} />
        <Stack.Screen name="Profile" component={ProfileScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

HOMESCREEN.JS

```
import React from 'react';
import { View, Text, Button } from 'react-native';

const HomeScreen = ({ navigation }) => (  // navigation prop is in
```

```
  <View style={{ flex: 1, justifyContent: 'center', alignItems: 'c
    <Text>Home Screen</Text>
    <Button
      title="Go to Profile"
      onPress={() => navigation.navigate('Profile')}  // navigate
    />
  </View>
);

export default HomeScreen;
```

MCQ Bank — All Topics

24 Multiple Choice Questions covering Weeks 10–12. Correct answers are highlighted in green.

WEEK 10 — REACT NATIVE INTRO

WEEK 10

Q1. React Native is primarily used for building:

- Web applications
- **Native mobile applications ✓**
- Desktop applications
- Server-side applications

WEEK 10

Q2. Which component in React Native is equivalent to the HTML ?

- Text
- Section
- **View ✓**
- Container

WEEK 10

Q3. What command is used to create a new Expo project?

- npm create expo
- **npx create-expo-app@latest ✓**

expo new project

react-native init

WEEK 10

Q4. Which tool is best for beginners who want to start React Native quickly?

- React Native CLI
- **Expo CLI ✓**
- Android Studio
- Xcode

WEEK 10

Q5. React Native converts JavaScript code into native code using which mechanism?

- Transpiler
- Compiler
- **Bridge Architecture ✓**
- Virtual Machine

WEEK 10

Q6. React Native CLI gives you access to which of the following?

- Expo Go app scanning
- **Full native code (Java/Swift) ✓**
- Automatic updates
- Cloud building

WEEK 11 — COMPONENTS & USER INPUT

WEEK 11

Q7. **In React Native, all visible text must be placed inside which component?**

- View
- **Text ✓**
- Paragraph
- Label

WEEK 11

Q8. **Which prop of TextInput fires a function every time the user types a character?**

- onChange
- **onChangeText ✓**
- onType
- onInput

WEEK 11

Q9. **What problem does KeyboardAvoidingView solve?**

- Makes keyboard appear faster
- Hides the keyboard
- **Prevents keyboard from covering input fields ✓**
- Changes keyboard language

WEEK 11

Q10. **What visual feedback does TouchableOpacity give when pressed?**

- The element shakes
- **The element becomes semi-transparent ✓**

The element turns red

The element disappears

WEEK 11

Q11. How is styling done in React Native (not CSS files)?

Using .css files

Using SASS

Using JavaScript objects with StyleSheet.create() ✓

Using inline HTML style attribute

WEEK 11

Q12. TypeScript files in React Native use which file extension for components?

.js

.ts

.tsx ✓

.jsx

WEEK 11 P2 — FLEXBOX

WEEK 11 P2

Q13. What is the default value of flexDirection in React Native?

row

column ✓

row-reverse

column-reverse

WEEK 11 P2

Q14. justifyContent aligns items along which axis?

- Cross Axis
- **Main Axis ✓**
- Z Axis
- Both axes equally

WEEK 11 P2

Q15. Three views have flex values 1, 2, 1 inside a flex: 1 parent. What fraction of height does the middle view get?

- 1/3
- **2/4 (50%) ✓**
- 1/4
- 2/3

WEEK 11 P2

Q16. Which property aligns a single child independently without affecting its siblings?

- justifyContent
- alignItems
- **alignSelf ✓**
- flex

WEEK 11 P2

Q17. What does flexWrap: 'wrap' do?

- Makes text bold
- **Moves items to the next line when space is full ✓**
- Reverses item order
- Stretches items to fill the

container

WEEK 11 P2

Q18. What is the purpose of SafeAreaView?

- Adds a security layer to the app
- **Keeps content away from notches and system UI elements ✓**
- Prevents screen rotation
- Locks the keyboard

WEEK 12 — SCROLLVIEW, FLATLIST & NAVIGATION

WEEK 12

Q19. What is the main difference between ScrollView and FlatList?

- ScrollView is faster
- **FlatList only renders visible items; ScrollView renders all at once ✓**
- FlatList can only scroll horizontally
- They are the same component

WEEK 12

Q20. Which required prop in FlatList provides a unique identifier for each item?

renderItem

data

keyExtractor ✓

getItemLayout

WEEK 12

Q21. Stack Navigation works like:

A carousel of screens

Tabs at the bottom

A stack of cards where new screens go on top ✓

A drawer that slides in

WEEK 12

Q22. What must wrap the entire app when using React Navigation?

Stack.Navigator

NavigationContainer ✓

SafeAreaView

NavigationWrapper

WEEK 12

Q23. Which FlatList prop limits how many items are rendered when the list first loads?

windowSize

maxToRenderPerBatch

initialNumToRender ✓

removeClippedSubviews

WEEK 12

Q24. To navigate back to the previous screen, which function do you call?

- navigation.navigate('Back')
- **navigation.goBack() ✓**
- navigation.pop()
- navigation.return()

QUIZZES

4 Quizzes — 10 Marks Each

Quiz 1 — React Native Introduction (Week 10)

Total: 10 Marks

1 Define React Native. How is it different from a web application built with React.js? *(explain at least two differences)* [2 marks]

> **Key points:** Cross-platform mobile framework; React.js → browser using HTML/CSS; React Native → phone using View/Text/native components.

---

2 What is Expo CLI? Write the four terminal commands needed to create, navigate into, start, and preview a new Expo project. [2 marks]

> **Commands:** `npx create-expo-app@latest` → `cd MyApp` → `npx expo start` → scan QR with Expo Go.

---

3 Compare Expo CLI and React Native CLI using any four features in a table format. [2 marks]

> **Hint:** Setup difficulty, native code access, testing method, best use case.

---

4 Write a complete React Native "Hello World" app that imports React, View and Text, and displays "My First App" on the screen. [2 marks]

> **Must include:** import statement, export default function, return with <View> wrapping <Text>.

---

5 What is "Hot Reloading" in React Native? Why is it useful for developers? [2 marks]

> **Answer:** Instantly updates the running app when code changes are saved — no restart needed; saves development time.

Quiz 2 — Basic Components & User Input (Week 11)

Total: 10 Marks

1 Name the four basic components of React Native discussed in Week 11, and write one line describing what each is used for. [2 marks]

> **Answer:** View (container), Text (display text), Image (show images), Button (clickable action).

---

2 Write a code snippet using TextInput and useState that captures a user's email address and displays it below the input field in real time. [3 marks]

> **Must include:** useState hook, TextInput with value, onChangeText, and a Text component showing the state value.

---

3 What is the difference between TouchableOpacity and Button? Give a scenario where TouchableOpacity is preferred. [2 marks]

> **Key difference:** Button appearance is OS-controlled; TouchableOpacity is fully customisable. Preferred when: custom styled button with icon needed.

---

4 Explain KeyboardAvoidingView. Which behavior value should you use for Android vs iOS? [2 marks]

> **Answer:** Prevents keyboard from overlapping inputs. Android → `'height'`; iOS → `'padding'`. Use Platform.OS to detect.

---

5 True or False — and explain: "In React Native you can display text by writing it directly inside a <View> without any <Text> [1 mark]

tag."

> **Answer:** FALSE. React Native will throw an error. All text must be wrapped in <Text> component.

Quiz 3 — FlexBox in React Native (Week 11 Part 2)

Total: 10 Marks

1 Explain the concept of Main Axis and Cross Axis in Flexbox. How do they change when you switch flexDirection from 'column' to 'row'? [2 marks]

> **Answer:** column → Main=vertical, Cross=horizontal. row → Main=horizontal, Cross=vertical. justifyContent always controls Main Axis.

---

2 A parent View has flex: 1 and three child Views with flex values 2, 1, 3. What percentage of the height does each child occupy? [2 marks]

> **Answer:** Total = 6. Child 1 = 2/6 ≈ 33%, Child 2 = 1/6 ≈ 17%, Child 3 = 3/6 = 50%.

---

3 Write the code for a View that perfectly centres a 100×100 blue box both horizontally and vertically on the screen. [2 marks]

> **Answer:** Parent needs: flex:1, justifyContent:'center', alignItems:'center'. Child: width:100, height:100, backgroundColor:'blue'.

---

4 Explain the difference between alignItems and alignSelf. When would you use alignSelf? [2 marks]

> **Answer:** alignItems applies to ALL children; alignSelf overrides it for ONE specific child. Use alignSelf when one element needs different cross-axis alignment.

---

5 What is SafeAreaView and why is it needed on modern smartphones? [2 marks]

> **Answer:** It adds automatic padding to avoid notches, status bars, home indicators, and rounded corners. Without it, content may hide behind system UI elements.

Quiz 4 — ScrollView, FlatList & Navigation (Week 12)

Total: 10 Marks

1 What is ScrollView? Write a complete ScrollView example that renders 20 items in a vertical list. [2 marks]

> **Key points:** Renders all children at once. Wrap items in <ScrollView>, use Array.from + .map to generate items.

---

2 Explain why FlatList is preferred over ScrollView for large data sets. What technique does it use internally? [2 marks]

> **Answer:** FlatList uses windowing/lazy loading — only renders visible items. ScrollView loads everything at once, wasting memory when list is large.

---

3 List and briefly explain any three FlatList performance optimisation props. [3 marks]

> **Any 3 from:** initialNumToRender, maxToRenderPerBatch, windowSize, removeClippedSubviews, getItemLayout — with correct description.

---

4 Describe Stack Navigation with an analogy. Write the App.js code to set up a Stack Navigator with two screens: Home and Settings. [3 marks]

> **Analogy:** Stack of cards. New screen goes on top, Back removes top card. Code: NavigationContainer → Stack.Navigator → Stack.Screen × 2.

📌 Final Term — Paper Questions

Weight: 40%

These are exam-style questions written from the perspective of what would appear in a final term paper. They test understanding, application, and analysis — not just memorisation. Study the definitions and code examples above to answer these confidently.

SECTION A — SHORT ANSWER QUESTIONS (2 MARKS EACH)

Q1 Define React Native and state two of its key features that make it popular for mobile app development. [2 marks]

Q2 What is the difference between `flexDirection: 'column'` and `flexDirection: 'row'` in React Native Flexbox? [2 marks]

Q3 State the purpose of `keyExtractor` prop in FlatList. What type of value should it return? [2 marks]

Q4 What are the three required props of a FlatList component? Explain each in one line. [2 marks]

Q5 Why do we use `StyleSheet.create()` instead of plain JavaScript objects for styling in React Native? [2 marks]

Q6 What is the `navigation` prop in a React Navigation screen component? How does it reach the screen component automatically? [2 marks]

SECTION B — EXPLANATION & COMPARISON QUESTIONS (4–5 MARKS EACH)

Q7 Compare Expo CLI and React Native CLI in detail. Your answer should cover: setup difficulty, native code access, testing method, best use case, and whether it can use Expo Go. (Tabular format is acceptable.) [5 marks]

Q8 Explain the Main Axis and Cross Axis concept in Flexbox. Using this understanding, explain how `justifyContent` and `alignItems` behave differently when `flexDirection` is changed from `'column'` to `'row'`. [5 marks]

Q9 Differentiate between `ScrollView` and `FlatList` in React Native. Under what conditions would you choose one over the other? Discuss performance implications for a list of 10,000 items. [4 marks]

Q10 Explain Stack Navigation in React Native with an appropriate real-world analogy. Describe what happens to the navigation stack when: (a) the user opens a new screen, (b) the user presses the Back button. [4 marks]

Q11 What problem does `KeyboardAvoidingView` solve? Explain the difference between using `behavior="padding"` and `behavior="height"`, and when each should be applied. [4 marks]

SECTION C — CODE WRITING QUESTIONS (5–10 MARKS EACH)

Q12 Write a React Native component that contains: (a) A TextInput for the user's name, (b) A TextInput for the user's email, (c) A TouchableOpacity styled button that, when pressed, shows an Alert with both values. Use useState hooks and StyleSheet.create for styling. [8 marks]

Q13 Write a complete React Native screen using Flexbox that creates a layout with: (a) a header (flex: 1, blue), (b) a two-column card row for "Students" and "Courses" statistics (flex: 2), (c) a 2×2 grid of menu buttons using flexWrap (flex: 4), and (d) a footer (flex: 1, dark). Use StyleSheet.create. [10 marks]

Q14 Write the complete code for a React Native app with two screens (HomeScreen and DetailScreen) connected using Stack Navigation. HomeScreen should display a FlatList of 5 items (id, title). When any item is pressed, it should navigate to DetailScreen passing the item title as a route parameter, which DetailScreen then displays. [10 marks]

Q15 Write an optimised FlatList that renders a dataset of 500 user objects (id, name, email). Include the following optimisation props and explain the role of each: `initialNumToRender`, `maxToRenderPerBatch`, `windowSize`, `removeClippedSubviews`, and `getItemLayout`. [8 marks]

SECTION D — SCENARIO / APPLICATION QUESTIONS (5 MARKS EACH)

Q16 Your friend is building a food delivery app. They ask: "Should I use ScrollView or FlatList for the restaurant menu which can have up to 500 items?" Give a justified recommendation with technical reasoning related to memory management and rendering. [5 marks]

Q17 A student's Expo app looks fine on iPhone but the content is hidden behind the camera notch on a newer Android phone. What component [5 marks]

should they use and why? Provide the corrected code structure.

Q18 A student writes: `<View>Hello World</View>` and the app crashes. Explain why this happens and write the corrected code. Also explain two other common React Native beginner mistakes with Flexbox. [5 marks]

Exam Tip: The final term carries 40% weight. Focus especially on: (1) Flexbox properties and the Main/Cross Axis concept, (2) FlatList vs ScrollView comparison, (3) Stack Navigation code structure, and (4) TextInput with useState. These appear most frequently as code-writing questions.